

Auth Service — Documentation technique

Aucune librairie d'authentification, de rate limiting, de session ou de PoW n'est utilisée. Tout est implémenté à la main avec le module natif crypto de Node.js.

Table des matières

1. Vue d'ensemble
 2. Architecture du projet
 3. JWT maison (HMAC-SHA256)
 4. Hachage de mot de passe (scrypt)
 5. Double Pepper (microservices externes)
 6. Rotation de token (refresh token)
 7. Rate Limiter basé sur le token
 8. Proof of Work (PoW)
 9. Authentification inter-microservices
 10. Schéma de base de données
 11. Endpoints API
 12. Installation et lancement
 13. Variables d'environnement
 14. Exemples d'utilisation (cURL)
 15. Mot de passe oublié (forgot / reset)
-

1. Vue d'ensemble

Le **Auth Service** est un microservice d'authentification **zéro librairie auth** pour la plateforme **DANEBCYS** :

Ce qu'on fait	Ce qu'on n'utilise PAS
JWT signé à la main (HMAC-SHA256)	jsonwebtoken
Rate limiter maison (par token)	express-rate-limit
PoW maison (challenge HMAC)	toute lib PoW
Gestion de session maison	toute lib de session

Stack :

- Express.js
 - PostgreSQL avec driver natif pg
 - crypto.scrypt pour le hachage
 - Nodemailer pour les emails (expéditeur : **DANEBCYS**)
-

2. Architecture du projet

```

Auth-service/
├─ src/
|   ├─ config/
|   |   ├─ database.js          # Pool PostgreSQL natif (pg) + initDB()
|   |   └─ env.js              # Variables d'environnement (validation)
|   ├─ controllers/
|   |   └─ auth.controller.js   # Handlers HTTP (validation entrée)
|   ├─ middlewares/
|   |   ├─ auth.js             # Vérification JWT maison
|   |   ├─ rateLimiter.js      # Rate limiter maison (par token)
|   |   ├─ pow.js              # Proof of Work maison (HMAC)
|   |   └─ serviceAuth.js      # Auth inter-microservices
|   ├─ routes/
|   |   ├─ auth.routes.js      # Routes publiques /api/v1/auth
|   |   └─ internal.routes.js  # Routes internes /internal
|   ├─ services/
|   |   ├─ auth.service.js     # Logique métier (signup, login, reset,
etc.)
|   |   └─ mail.service.js     # Nodemailer (expéditeur : DANEBCYS)
|   |   └─ pepper.service.js   # Double pepper (primary + secondary,
externes)
|   |       └─ token.service.js # JWT maison + rotation refresh tokens
|   ├─ utils/
|   |   ├─ jwt.js              # Implémentation JWT HMAC-SHA256
|   |   └─ hash.js            # script + timing-safe compare +
fingerprint
|   |       └─ errors.js       # Classes d'erreur (AppError, 400-429)
|   └─ app.js                 # Config Express (helmet, cors, routes,
error handler)
├─ public/                   # Frontend de test
|   ├─ index.html            # Interface de test (navigateur)
|   └─ test.js               # Logique client (PoW solver, appels API)
├─ init.sql                  # Schéma PostgreSQL (roles, users,
sessions, etc.)
├─ package.json
├─ server.js                 # Point d'entrée (connect DB → initDB →
initPepper → listen)

```

```

pepper-primary/                                # Pepper primaire (HORS du dossier Auth-
service)
├─ server.js                                    # Serveur HTTP natif (http.createServer)
├─ package.json
└─ .env

pepper-service/                                # Pepper secondaire (HORS du dossier
Auth-service)
├─ server.js                                    # Serveur HTTP natif (http.createServer)
├─ package.json
└─ .env

```

3. JWT maison (HMAC-SHA256)

Fichier : src/utils/jwt.js

On a implémenté le standard JWT (RFC 7519) uniquement avec crypto natif :

Structure d'un token

```
HEADER.PAYLOAD.SIGNATURE
```

Chaque partie est encodée en **Base64URL** (pas Base64 classique) :

```

Header:    { "alg": "HS256", "typ": "JWT" }
Payload:   { "sub": "user-uuid", "email": "...", "role": "user", "iat":
..., "exp": ... }
Signature: HMAC-SHA256(header_b64 + "." + payload_b64, secret)

```

Création (sign)

1. Encoder le header en Base64URL
2. Ajouter iat (issued at) et exp (expiration) au payload → exp calculé via `parseDurationSec()` : supporte "15m", "7d", "1h", etc.
3. Encoder le payload en Base64URL
4. Calculer HMAC-SHA256 sur "header.payload" avec le secret
5. Encoder la signature en Base64URL
6. Retourner "header.payload.signature"

Vérification (verify)

1. Séparer le token en 3 parties
2. Recalculer HMAC-SHA256 sur "header.payload"

3. Comparer avec la signature reçue via `crypto.timingSafeEqual()` : empêche les attaques par timing (constant-time comparison)
4. Décoder le payload, vérifier que `exp > maintenant`
5. Retourner le payload

Décodage sans vérification (decode)

1. Séparer le token en 3 parties
2. Décoder le payload Base64URL en JSON
3. Retourner le payload (sans vérifier la signature) : Utilisé uniquement par le rate limiter pour extraire le user ID

Pourquoi `timingSafeEqual` ?

Une comparaison classique (`= == =`) s'arrête au premier octet différent. Un attaquant pourrait mesurer le temps de réponse pour deviner la signature octet par octet. `timingSafeEqual` compare TOUS les octets en temps constant.

4. Hachage de mot de passe (scrypt)

Fichier : `src/utls/hash.js`

Pourquoi pas `bcrypt` ?

- `bcrypt` est une dépendance **native C++** (problèmes de compilation)
- `bcrypt` ne résiste pas aussi bien aux attaques **GPU/ASIC**
- `crypto.scrypt` est **intégré à Node.js**, zéro dépendance

Pourquoi `scrypt` ? `scrypt` est une fonction de dérivation de clé **memory-hard** : elle nécessite beaucoup de **mémoire** en plus du CPU. Cela rend les attaques par GPU/ASIC très coûteuses car ces composants ont peu de mémoire par cœur.

Paramètres

Paramètre	Valeur	Rôle
N	16384	Coût CPU/mémoire
r	8	Taille de bloc
p	1	Parallélisme
keyLength	64	Taille de la clé dérivée

Flux de hachage

1. Générer un salt aléatoire de 32 octets (`crypto.randomBytes`)
2. Concaténer le mot de passe + le double pepper combiné
3. Appliquer `scrypt(input, salt, 64)` avec les paramètres ci-dessus
4. Retourner { salt, hash } séparément : Stockés dans deux colonnes dédiées en DB (salt et `password_hash`)

Flux de vérification

1. Récupérer le salt et le password_hash depuis la DB
2. Recalculer `script(password + pepper, salt, 64)`
3. Comparer avec `crypto.timingSafeEqual()` (pas de `===`)

Fonctions utilitaires

- **safeCompare(a, b)** : comparaison timing-safe universelle. Hash les deux valeurs en SHA-256 avant de comparer pour garantir des buffers de même taille, quelle que soit la longueur des entrées.
 - **createFingerprint(userAgent)** : crée un hash SHA-256 du User-Agent pour le fingerprinting de session.
-

5. Double Pepper (microservices externes)

Fichiers : `src/services/pepper.service.js` + `pepper-primary/` + `pepper-service/`

Concept : Le **pepper** est un secret cryptographique ajouté au mot de passe avant le hachage. Contrairement au salt (stocké en DB), le pepper est **hors de la base de données**.

On utilise un **double pepper entièrement externe**, le Auth Service ne stocke aucun pepper :

Pepper	Source	Stockage
Primaire	pepper-primary (microservice)	.env du pepper-primary
Secondaire	pepper-service (microservice)	.env du pepper-service

Les deux microservices sont des dossiers **séparés**, au même niveau que le Auth Service.

Combinaison:

```
combined_pepper = HMAC-SHA256(PRIMARY_PEPPER, SECONDARY_PEPPER)
```

On utilise HMAC (et pas une simple concaténation) pour que le pepper combiné soit une valeur dérivée cryptographiquement forte, indépendante de la longueur des entrées.

Les deux Pepper Services: Chaque service suit la même architecture :

- Serveur **HTTP natif** (`http.createServer`), pas d'Express
- Protégé par X-Service-Key (SHA-256 + timing-safe comparison)
- Endpoint unique : `GET /pepper`
- Endpoint de santé : `GET /health`
- En production : réseau interne, pas d'accès public

Service	Port par défaut	Clé Auth Service
pepper-primary	3098	PEPPER_PRIMARY_KEY
pepper-service	3099	PEPPER_SECONDARY_KEY

Avantage sécurité : Le Auth Service ne contient **aucun pepper**. Un attaquant doit compromettre :

- La base de données : il a les hashes mais aucun pepper
- Le .env du Auth Service : il n'a **aucun pepper** (juste les URLs et clés de service)
- Le .env du pepper-primary : il a le primaire mais pas le secondaire
- Le .env du pepper-service : il a le secondaire mais pas le primaire

Il faut compromettre **les trois services** pour reconstituer le pepper complet.

Mode développement : Si aucun PEPPER_PRIMARY_URL / PEPPER_SECONDARY_URL n'est configuré, ou si les services sont inaccessibles, le Auth Service utilise un fallback déterministe (sha256('dev-fallback-pepper')) uniquement en NODE_ENV=development. En production, les deux services sont obligatoires.

Dégradation gracieuse : Si un seul pepper est disponible, le service fonctionne avec un seul pepper hashé en SHA-256. Un avertissement est loggé.

6. Rotation de token (refresh token)

Fichier : src/services/token.service.js

Concept : À chaque utilisation d'un refresh token, l'ancien est **marqué comme invalidé** et un nouveau est émis. Si un token invalidé est réutilisé, c'est une **attaque par vol de token** → toutes les sessions du user sont détruites.

Génération :

- **Access Token :** JWT signé HMAC-SHA256 contenant { sub, email, role }, durée 15 min
- **Refresh Token :** 40 octets aléatoires (crypto.randomBytes(40).toString('hex'))

Stockage : Les refresh tokens sont stockés **hashés** en SHA-256 dans la table sessions (colonne token_hash). Même si la DB est compromise, les tokens bruts ne sont pas exposés. Chaque session stocke aussi le fingerprint_hash (User-Agent hashé) et l'ip_address.

Flux normal :

1. User se connecte : reçoit Access Token (15 min) + Refresh Token T1
2. Access Token expire : User envoie T1 à POST /refresh
3. Serveur : T1 existe et n'est PAS invalidated : Marquer T1 comme invalidated (UPDATE sessions SET invalidated = TRUE) : Générer et retourner T2 (nouveau refresh) + nouvel Access Token

4. Plus tard, User envoie T2 : même processus : T3, etc.

Détection de vol :

1. Attaquant vole T1
2. Le vrai User utilise T1 : reçoit T2 (T1 est marqué invalidated)
3. L'Attaquant utilise T1 (déjà invalidated !) : DÉTECTION : réutilisation d'un token invalidé : DELETE toutes les sessions du user (DELETE FROM sessions WHERE user_id = ...) : L'attaquant ET le user sont déconnectés : Le user doit se reconnecter (reset de sécurité) : Erreur TOKEN_REUSE_DETECTED propagée au client

Sécurités supplémentaires

- Token expiré : retourne null (pas d'erreur, juste invalide)
 - Utilisateur soft-deleted : retourne null
 - Déconnexion (logout) : DELETE FROM sessions WHERE token_hash = ...
 - Reset password → DELETE FROM sessions WHERE user_id = ... (toutes les sessions)
-

7. Rate Limiter basé sur le token

Fichier : src/middlewares/rateLimiter.js

Pourquoi basé sur le token et pas l'IP ?

- Derrière un proxy/VPN, des milliers d'utilisateurs partagent la même IP
- Un attaquant peut changer d'IP facilement (proxy rotating)
- Le token identifie l'**utilisateur réel**, pas l'infrastructure réseau

Implémentation Rate limiter maison en mémoire (Map), sans aucune librairie :

1. Extraire la clé d'identification via la fonction keyFn :
 - tokenLimiter : req.user.id (après auth JWT)
 - preAuthTokenLimiter : JWT decode sans vérification (sub)
 - challengeLimiter : IP du client (fallback)
2. Chercher/créer une entrée dans la Map : { count, resetAt }
3. Incrémenter le compteur
4. Si count > max : HTTP 429 Too Many Requests
5. Headers de réponse : X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
6. Nettoyage périodique des entrées expirées (setInterval 60s, .unref())

Trois niveaux :

Limiter	Clé	Cible	Config
tokenLimiter	req.user.id	Routes authentifiées (logout, me, verify-email)	100 req / 15 min
preAuthTokenLimiter	JWT decode (sub)	POST /refresh	100 req / 15 min
challengeLimiter	IP (fallback)	GET /pow-challenge	30 req / 1 min

Les routes signup/login/forgot-password/reset-password sont protégées par le **PoW** (pas de rate limiter, le PoW est la protection).

8. Proof of Work (PoW)

Fichier : src/middlewares/pow.js

Concept Le PoW force le client à effectuer un calcul coûteux avant chaque signup, login, forgot-password et reset-password. Cela empêche les attaques automatisées (bots, brute force).

Flux

1. Client : GET /api/v1/auth/pow-challenge : { challenge, difficulty, signature, expiresAt }
2. Client résout le puzzle : Trouver un nonce tel que SHA-256(challenge + nonce) commence par difficulty zéros hexadécimaux
3. Client : POST /api/v1/auth/signup (ou /login, /forgot-password, /reset-password)
Headers : x-pow-challenge: <challenge> x-pow-nonce: <nonce trouvé> x-pow-signature: <signature HMAC>

Création du challenge (serveur)

1. Générer timestamp + 16 octets aléatoires : "timestamp:random_hex"
2. Signer avec HMAC-SHA256(challenge, POW_SECRET)
3. Retourner { challenge, difficulty, signature, expiresAt }

Vérification serveur

1. Vérifier la signature HMAC du challenge (anti-forgery) : Recalculer HMAC-SHA256(challenge, POW_SECRET) : crypto.timingSafeEqual sur les buffers hex (timing-safe)
2. Vérifier que le challenge n'est pas expiré : Extraire le timestamp du challenge, comparer avec POW_EXPIRY_SECONDS
3. Vérifier SHA-256(challenge + nonce) commence par N zéros : '0'.repeat(difficulty) comme préfixe attendu

Pourquoi stateless ? Le challenge est signé par HMAC avec un secret serveur. Pas besoin de le stocker en DB. Le timestamp intégré dans le challenge (timestamp:random) permet de vérifier l'expiration. C'est plus efficace et élimine une table DB.

Difficulté Avec POW_DIFFICULTY=4, le client doit trouver un hash commençant par 0000. En moyenne, cela prend ~65 000 tentatives. C'est rapide pour un humain (navigateur moderne avec crypto.subtle.digest : quelques secondes) mais coûteux pour un bot qui fait des milliers de requêtes.

Résolution côté client Le frontend (public/test.js) résout le PoW avec l'API Web Crypto :

1. Récupérer le challenge via GET /pow-challenge
2. Boucle : nonce = 0, 1, 2, ... : SHA-256(challenge + nonce) via crypto.subtle.digest('SHA-256', ...) : Vérifier si le hash commence par le préfixe de zéros
3. Yield au navigateur toutes les 5000 tentatives (setTimeout(0))
4. Envoyer le nonce trouvé dans les headers de la requête

9. Authentification inter-microservices

Fichiers : src/middlewares/serviceAuth.js + src/routes/internal.routes.js

Concept Les autres microservices de DANEBCYS (Users Service, Ads Service, etc.) doivent pouvoir valider les tokens des utilisateurs sans connaître le secret JWT. Ils appellent le Auth Service via une route interne.

Middleware serviceAuth

1. Lire le header X-Service-Key
2. Comparer avec INTER_SERVICE_KEY via safeCompare() : safeCompare hash les deux valeurs en SHA-256 avant de comparer : crypto.timingSafeEqual sur les digests (buffers de taille fixe)
3. Si valide : req.isService = true : next()
4. Si absent : 401 (Header X-Service-Key manquant)
5. Si invalide : 403 (Clé de service invalide)

Route interne

```
POST /internal/validate-token
Header: X-Service-Key: <clé partagée>
Body: { "accessToken": "eyJhbG..." }

Réponse OK : { "valid": true, "user": { "id": "...", "email": "...",
"role": "..." } }
Réponse KO : { "valid": false, "error": "Token expiré" }
```

Le endpoint vérifie le JWT avec `jwt.verify()`, puis récupère l'utilisateur en DB (vérifie qu'il existe et n'est pas soft-deleted) et retourne ses informations enrichies (rôle depuis la table `roles`).

Flux inter-services

1. User envoie une requête au Users Service avec son Access Token
 2. Users Service appelle Auth Service : POST `/internal/validate-token` Headers: X-Service-Key: shared-secret Body: { accessToken: "..." }
 3. Auth Service vérifie le JWT et retourne le user
 4. Users Service continue le traitement
-

10. Schéma de base de données

Fichier : `init.sql` — exécuté automatiquement au démarrage via `initDB()`

Table roles

Colonne	Type	Description
id	SERIAL (PK)	Auto-increment
name	VARCHAR(50)	UNIQUE NOT NULL
description	TEXT	Description du rôle
created_at	TIMESTAMPTZ	Date de création
updated_at	TIMESTAMPTZ	Trigger auto-update

Rôles par défaut : user, vendeur, assistance, admin

Table users

Colonne	Type	Description
id	UUID (PK)	<code>gen_random_uuid()</code>
username	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL
phone	VARCHAR(20)	Optionnel
password_hash	VARCHAR(512)	scrypt hash (hex)
salt	VARCHAR(128)	Salt séparé (hex)
role_id	INTEGER (FK)	<code>roles.id</code> , défaut: 1 (user)
premium_level	<code>premium_level_enum</code>	'none', 'premium', 'premium_avancee', 'etudiant'
student_proof	VARCHAR(512)	Justificatif étudiant
country	VARCHAR(100)	Pays
first_name	VARCHAR(100)	Prénom
last_name	VARCHAR(100)	Nom
email_verified	BOOLEAN	Défaut: false
phone_verified	BOOLEAN	Défaut: false
deleted	BOOLEAN	Soft delete, défaut: false
created_at	TIMESTAMPTZ	Date de création
updated_at	TIMESTAMPTZ	Trigger auto-update

last_login_at	TIMESTAMPTZ	Dernière connexion
---------------	-------------	--------------------

Index partiels (soft delete) :

- idx_users_email_active : UNIQUE sur email WHERE deleted = FALSE
- idx_users_username_active : UNIQUE sur username WHERE deleted = FALSE
- idx_users_phone_active : UNIQUE sur phone WHERE deleted = FALSE AND phone IS NOT NULL

Table sessions

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	users.id ON DELETE CASCADE
token_hash	VARCHAR(512)	SHA-256 du refresh token
fingerprint_hash	VARCHAR(256)	SHA-256 du User-Agent
ip_address	VARCHAR(45)	IP du client
invalidated	BOOLEAN	Pour la détection de réutilisation
expires_at	TIMESTAMPTZ	Expiration (7 jours par défaut)
created_at	TIMESTAMPTZ	Date de création
last_used_at	TIMESTAMPTZ	Dernière utilisation

Table email_verifications

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	users.id ON DELETE CASCADE
code	VARCHAR(6)	Code à 6 chiffres (crypto.randomInt)
expires_at	TIMESTAMPTZ	15 min après création
used	BOOLEAN	Défaut: false
created_at	TIMESTAMPTZ	Date de création

Table password_resets

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	users.id ON DELETE CASCADE
code	VARCHAR(6)	Code à 6 chiffres (crypto.randomInt)
expires_at	TIMESTAMPTZ	15 min après création
used	BOOLEAN	Défaut: false
created_at	TIMESTAMPTZ	Date de création

Triggers

- **trg_users_updated_at** : BEFORE UPDATE sur users : updated_at = NOW()
- **trg_roles_updated_at** : BEFORE UPDATE sur roles : updated_at = NOW()

Extension requise

- pgcrypto : pour gen_random_uuid() (créée par init.sql)

11. Endpoints API

Routes publiques — /api/v1/auth

Méthode	Route	Protection	Description
GET	/pow-challenge	Rate limit (IP, 30/min)	Obtenir un challenge PoW
POST	/signup	PoW	Inscription
POST	/login	PoW	Connexion
POST	/forgot-password	PoW	Demande de réinitialisation
POST	/reset-password	PoW	Réinitialisation du mot de passe
POST	/refresh	Rate limit (token, 100/15min)	Rotation des tokens
POST	/logout	JWT + Rate (token)	Déconnexion
GET	/me	JWT + Rate (token)	Profil connecté
POST	/verify-email	JWT + Rate (token)	Vérification email

Routes internes — /internal

Méthode	Route	Protection	Description
POST	/validate-token	X-Service-Key	Validation de token pour les autres microservices

Routes utilitaires

Méthode	Route	Protection	Description
GET	/health	Aucune	Health check

12. Installation et lancement

Prérequis

- Node.js >= 18
- PostgreSQL >= 14

Auth Service

```
cd Auth-service
npm install

# Créer la base de données (si pas déjà fait)
```

```
psql -U postgres -c "CREATE DATABASE danebcys;"

# Développement (auto-reload avec --watch)
npm run dev

# Production
npm start
```

Le schéma PostgreSQL (init.sql) est exécuté **automatiquement** au démarrage du serveur via `initDB()`. Pas besoin de `npm run db:init` manuellement (mais la commande existe si besoin).

Pepper Services (microservices externes)

```
# Pepper primaire
cd pepper-primary
npm install
# Configurer .env : PORT, SERVICE_KEY, PEPPER_VALUE
npm start
# → [Pepper Primary] Port 3098

# Pepper secondaire
cd pepper-service
npm install
# Configurer .env : PORT, SERVICE_KEY, PEPPER_VALUE
npm start
# → [Pepper Service] Port 3099
```

Ordre de démarrage

1. **pepper-primary** d'abord (port 3098)
2. **pepper-service** ensuite (port 3099)
3. **Auth Service** en dernier (port 3001) — il récupère les deux peppers au démarrage

En mode dev, si les Pepper Services ne sont pas lancés, l'Auth Service utilise un fallback automatique.

Séquence de démarrage du Auth Service

1. Connexion au pool PostgreSQL (SELECT 1)
2. Initialisation du schéma DB (init.sql via `initDB()`)
3. Récupération des peppers (`initPepper()`) → appels HTTP aux Pepper Services)
4. Démarrage du serveur Express (`app.listen`)

13. Variables d'environnement

Auth Service (.env)

Variable	Description	Défaut
PORT	Port du serveur	3001
NODE_ENV	Environnement	development
PG_HOST	Hôte PostgreSQL	localhost
PG_PORT	Port PostgreSQL	5432
PG_DATABASE	Nom de la base	auth_service
PG_USER	Utilisateur PostgreSQL	postgres
PG_PASSWORD	Mot de passe PostgreSQL	postgres
JWT_ACCESS_SECRET	Secret HMAC pour les access tokens	requis
JWT_REFRESH_SECRET	Secret HMAC pour les refresh tokens	requis
JWT_ACCESS_EXPIRES_IN	Durée access token	15m
JWT_REFRESH_EXPIRES_IN	Durée refresh token	7d
PEPPER_PRIMARY_URL	URL du pepper-primary	requis en prod
PEPPER_PRIMARY_KEY	Clé partagée avec pepper-primary	requis en prod
PEPPER_SECONDARY_URL	URL du pepper-service	requis en prod
PEPPER_SECONDARY_KEY	Clé partagée avec pepper-service	requis en prod
EMAIL_USER	Adresse Gmail (expéditeur DANEBCYS)	requis
EMAIL_PASS	App password Gmail	requis
POW_DIFFICULTY	Zéros requis pour le PoW	4
POW_SECRET	Secret HMAC pour les challenges PoW	requis
POW_EXPIRY_SECONDS	Durée de validité d'un challenge	300
INTER_SERVICE_KEY	Clé pour l'auth inter-microservices	requis
RATE_LIMIT_WINDOW_MS	Fenêtre du rate limit (ms)	900000
RATE_LIMIT_MAX_REQUESTS	Max requêtes par fenêtre	100

pepper-primary (.env)

Variable	Description
PORT	Port du serveur (défaut: 3098)
SERVICE_KEY	Clé partagée (= PEPPER_PRIMARY_KEY)
PEPPER_VALUE	La valeur du pepper primaire

pepper-service (.env)

Variable	Description
PORT	Port du serveur (défaut: 3099)
SERVICE_KEY	Clé partagée (= PEPPER_SECONDARY_KEY)
PEPPER_VALUE	La valeur du pepper secondaire

14. Exemples d'utilisation (cURL)

Health check

```
curl http://localhost:3001/health
```

Obtenir un challenge PoW

```
curl http://localhost:3001/api/v1/auth/pow-challenge
```

Inscription

```
curl -X POST http://localhost:3001/api/v1/auth/signup \
  -H "Content-Type: application/json" \
  -H "x-pow-challenge: CHALLENGE" \
  -H "x-pow-nonce: NONCE" \
  -H "x-pow-signature: SIGNATURE" \
  -d '{
    "username": "jeandupont",
    "email": "jean@example.com",
    "password": "motDePasse123",
    "firstName": "Jean",
    "lastName": "Dupont",
    "phone": "+33612345678",
    "country": "France"
  }'
```

Connexion

```
curl -X POST http://localhost:3001/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -H "x-pow-challenge: CHALLENGE" \
  -H "x-pow-nonce: NONCE" \
  -H "x-pow-signature: SIGNATURE" \
  -d '{ "email": "jean@example.com", "password": "motDePasse123" }'
```

Profil

```
curl http://localhost:3001/api/v1/auth/me \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Rafraîchir les tokens (rotation)

```
curl -X POST http://localhost:3001/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{ "refreshToken": "REFRESH_TOKEN" }'
```

Vérifier l'email

```
curl -X POST http://localhost:3001/api/v1/auth/verify-email \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{ "code": "482619" }'
```

Déconnexion

```
curl -X POST http://localhost:3001/api/v1/auth/logout \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{ "refreshToken": "REFRESH_TOKEN" }'
```

Mot de passe oublié

```
curl -X POST http://localhost:3001/api/v1/auth/forgot-password \
  -H "Content-Type: application/json" \
  -H "x-pow-challenge: CHALLENGE" \
  -H "x-pow-nonce: NONCE" \
  -H "x-pow-signature: SIGNATURE" \
  -d '{ "email": "jean@example.com" }'
```

Réinitialiser le mot de passe

```
curl -X POST http://localhost:3001/api/v1/auth/reset-password \
  -H "Content-Type: application/json" \
  -H "x-pow-challenge: CHALLENGE" \
  -H "x-pow-nonce: NONCE" \
  -H "x-pow-signature: SIGNATURE" \
  -d '{
    "email": "jean@example.com",
```



```
"code": "482619",  
"newPassword": "nouveauMotDePasse123"  
}'
```

Validation inter-services

```
curl -X POST http://localhost:3001/internal/validate-token \  
-H "Content-Type: application/json" \  
-H "X-Service-Key: votre-cle-inter-service" \  
-d '{ "accessToken": "ACCESS_TOKEN" }'
```

Interface de test Ouvrir <http://localhost:3001> dans le navigateur. Le frontend résout automatiquement le PoW côté client via `crypto.subtle.digest('SHA-256', ...)` et permet de tester tous les endpoints graphiquement.

15. Mot de passe oublié (forgot / reset)

Fichiers : `src/services/auth.service.js` + `src/services/mail.service.js`

Flux

1. User appelle `POST /forgot-password` avec `{ email }` → Le serveur **NE RÉVÈLE PAS** si l'email existe (toujours même réponse) → Si l'email existe : génère un code 6 chiffres (`crypto.randomInt`), stocke dans `password_resets`, envoie par email (expéditeur : `DANEBCYS`) → Le code expire après 15 minutes
2. User reçoit l'email de `DANEBCYS` avec le code
3. User appelle `POST /reset-password` avec `{ email, code, newPassword }` → Le serveur vérifie le code (non utilisé, non expiré, user non deleted) → Hash le nouveau mot de passe avec `scrypt` + double pepper → Met à jour `password_hash` et `salt` en DB → Révoque TOUTES les sessions du user (`DELETE FROM sessions WHERE user_id = ...`)

Sécurité

- **Anti-énumération** : `POST /forgot-password` retourne toujours le même message, qu'il y ait un compte ou non
 - **PoW obligatoire** : empêche le spam d'emails de réinitialisation
 - **Code à usage unique** : marqué `used = TRUE` après utilisation
 - **Révocation totale** : après un reset, toutes les sessions actives sont détruites (au cas où le mot de passe a été compromis)
 - **Validation du mot de passe** : minimum 8 caractères (vérifié par le controller)
-

Récapitulatif sécurité

Mesure	Implémentation
JWT	HMAC-SHA256 maison (crypto), timingSafeEqual
Hachage mot de passe	scrypt (memory-hard) + salt unique + double pepper externe
Double pepper	Deux microservices HTTP natifs séparés, combinés via HMAC
Rotation refresh token	Invalidation + détection de réutilisation (vol de token)
Fingerprint de session	SHA-256 du User-Agent stocké avec la session
Rate limiting	Maison, basé sur le token (user ID), pas l'IP
Proof of Work	Challenge HMAC stateless, SHA-256 avec difficulté ajustable
Auth inter-services	X-Service-Key + SHA-256 timingSafeEqual
Anti-timing attack	crypto.timingSafeEqual partout (JWT, passwords, clés)
Injection SQL	Requêtes paramétrées (\$1, \$2, ...) exclusivement
Headers sécurité	helmet (CSP, X-Frame-Options, etc.)
Soft Delete	Champ deleted + index partiels WHERE deleted = FALSE
Refresh token hashé	SHA-256 en DB, jamais en clair
Reset password	Code 6 chiffres, 15 min, usage unique, révocation totale
Anti-énumération	forgot-password ne révèle pas si l'email existe
Rôles RBAC	Table roles séparée (user, vendeur, assistance, admin)
Extraction IP client	Support X-Client-IP, X-Forwarded-For, req.ip
Emails	Expéditeur DANEBBCYS via Nodemailer (Gmail)

Librairies auth utilisées : aucune.